


# Retail Sales Data Warehouse

Data Catalog & Project References

roqiua mohammad 10/15/26

# Contents

**Business Reequipments:** ..... 2

**Task Requirements** ..... **Error! Bookmark not defined.**

**Data Sources:** ..... 2

**Data Integrations (How tables are related):** ..... 4

**Data Architecture:** ..... 5

- Architecture Diagram: ..... 5
- Data Flow: ..... 6

**ETL Pipeline:** ..... 6

**Execute Package:** ..... 12

**Example Table Package** ..... 12

**Example Log Batch:** ..... 13

**Data Layers:** ..... 23

- Stage Layer: ..... 23
  - Data Dictionary: ..... 23
  - Tables: ..... 24

**Technologies Used:** ..... 27

**Links & References:** ..... 27

## Project Overview:

Design and implement a retail data warehouse (RetailDWH) using SQL Server and SSIS to integrate, transform, and analyze sales data through a star schema with four dimensions and one fact table, supporting reliable reporting and business decision-making.

## Business Reequipments:

1. Design and implement a centralized data warehouse named RetailDWH for retail sales analysis.
2. Integrate data from the SQL Server operational database and product catalog file.
3. Create a staging layer to store and prepare source data before loading the warehouse.
4. Design a star schema consisting of four-dimension tables and one sales fact table.
5. Clean, validate, and transform data while ensuring data quality and referential integrity.
6. Track historical changes in relevant dimensions, particularly product attributes.
7. Implement an SSIS ETL pipeline to automate data extraction, transformation, and loading.
8. Support full and incremental loading, with error handling and ETL execution logging.
9. Enable sales analysis by date, customer, store, and product to support business decision-making.

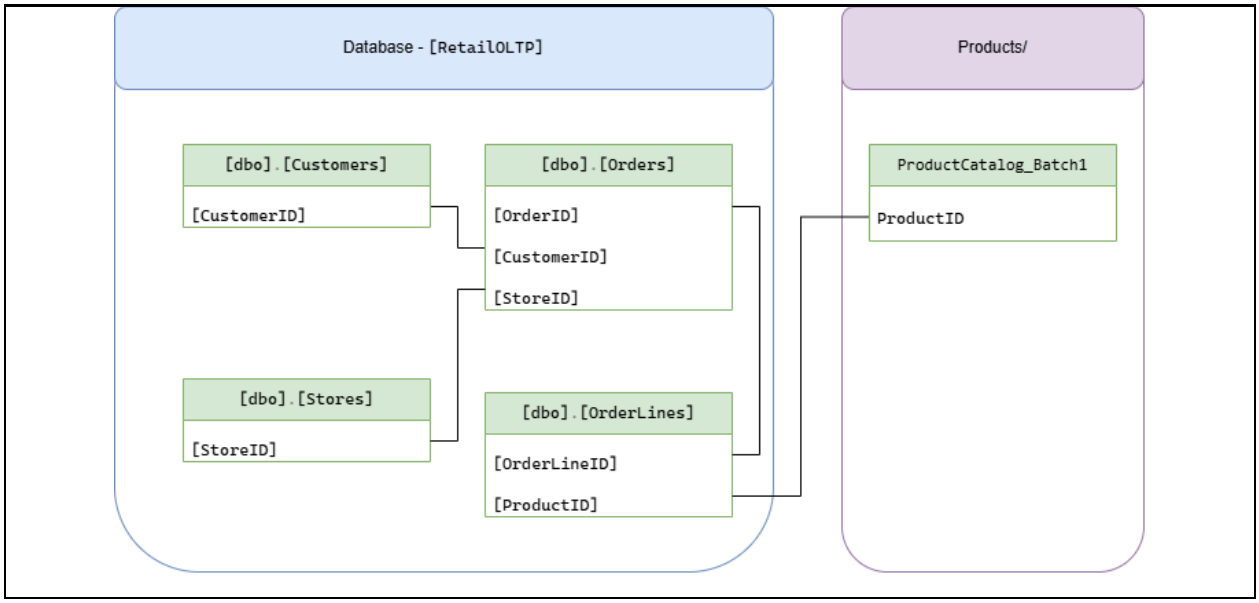
## Data Sources:

**Total: 5 Sources**

| **1. [dbo].[Customers]** | |
|---|---|
| **Source Type** | Database – Sql Server |
| **Source Location** | [RetailOLTP] |
| **Description** | Customer master data, including names and contact details. |
| | |
| **2. [dbo].[Orders]** | |
| **Source Type** | Database – Sql Server |
| **Source Location** | [RetailOLTP] |

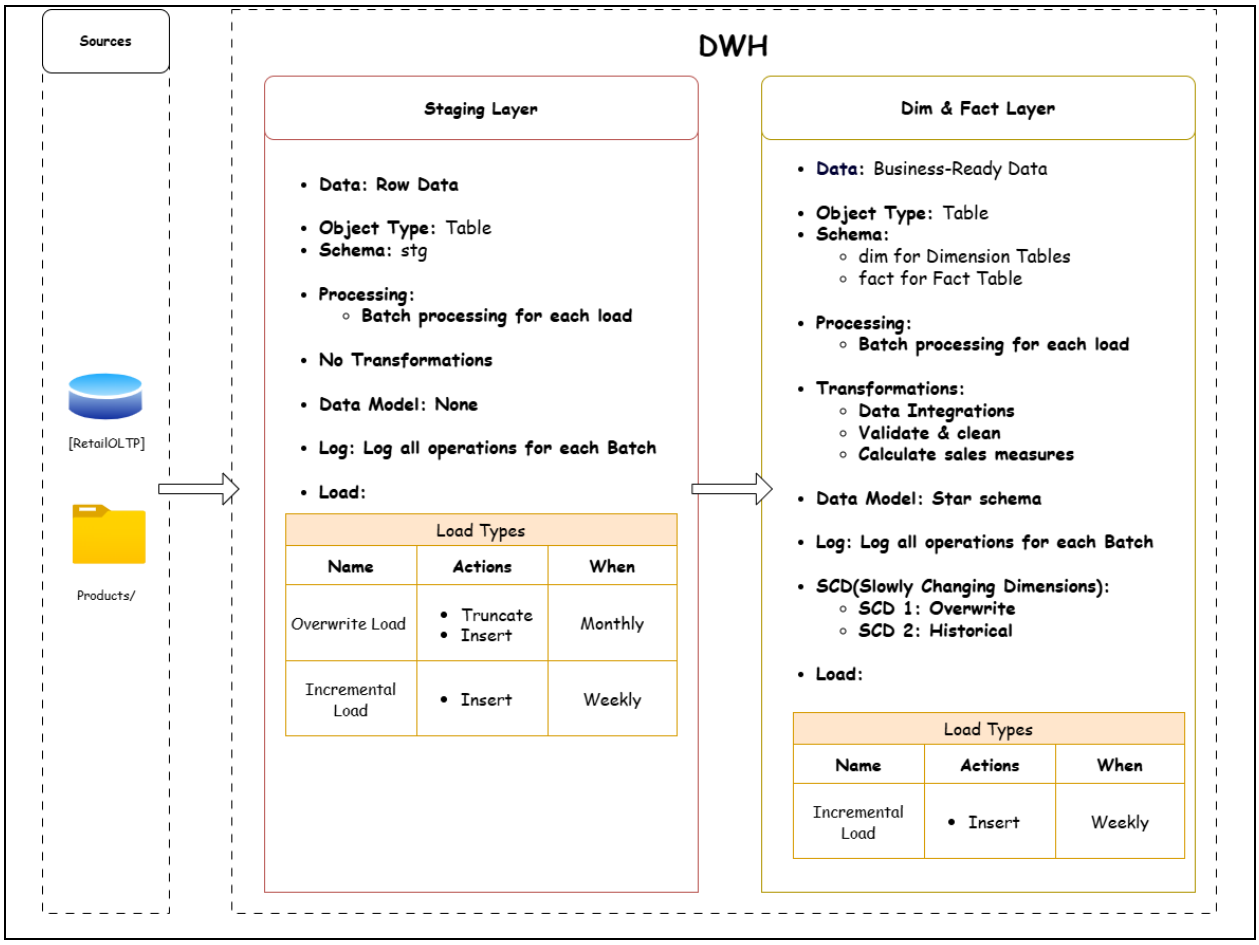
| | |
|---|---|
| **Description** | Sales order header information, including customer, store, order date, and status. |
| | |
| **3. [dbo].[OrderLines]** | |
| **Source Type** | Database – Sql Server |
| **Source Location** | [RetailOLTP] |
| **Description** | Sales transaction details, including products, quantities, prices, and discounts. |
| | |
| **4. [dbo].[Stores]** | |
| **Source Type** | Database – Sql Server |
| **Source Location** | [RetailOLTP] |
| **Description** | Store master data, including store names and locations. |
| | |
| **5. ProductCatalog_Batch1** | |
| **Source Type** | Excel |
| **Source Location** | /Products |
| **Description** | Sales transaction details, including products, quantities, prices, and discounts. |

## Data Integrations (How tables are related):

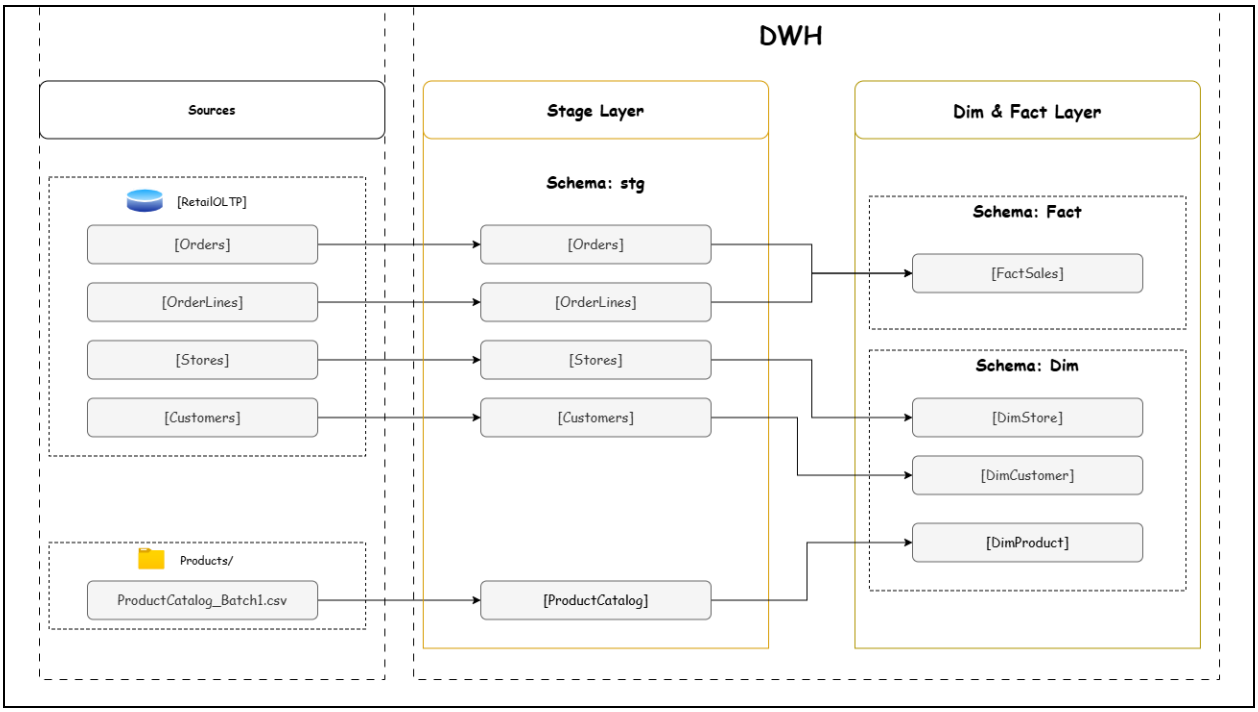


# Data Architecture:

- ## Architecture Diagram:



- Data Flow:




## ETL Pipeline Design and Loading Strategy:

**1. Staging Layer Loading Strategy**

The staging layer supports two loading strategies: **Overwrite (Monthly)** and **Incremental (Weekly)**.

No transformations, updates, or transactions are performed on the staging data. The layer retains the source data as received.

No inferred members are detected in scd1 or created in the staging layer.

**1.1 Overwrite Load (Monthly)**

- **Truncate:** The target staging table is cleared before loading.
- **Insert:** All available source data is reloaded into the staging table.

**1.2 Incremental Load (Weekly)**

- Only new or modified records are extracted based on the LastModifiedAt column.
- **SCD Type 1** is applied to all staging tables to insert new records and overwrite existing records when changes are detected.

**2. Dimension and Fact Loading Strategy**

The Dimension and Fact layers support **Incremental Loading only**.

The loading process includes data cleansing, transformation, data quality validation, and loading into the target tables.

Only new or modified records are extracted based on the **last successful watermark recorded in the Watermarks table**.

**2.1 Dimension Loading**

- **SCD Type 1:** Applied to all dimension tables except DimProduct, updating existing records without preserving historical changes.
- **SCD Type 2:** Applied to DimProduct to maintain historical product versions, particularly changes in product prices. This ensures that sales transactions are associated with the correct product cost for the relevant period, supporting accurate profit calculations and preventing historical data conflicts.
- The selection of product version StartDate and EndDate values will be explained in the Data Dictionary.

**2.2 Fact Loading**

The fact table is loaded incrementally after the required dimensions are processed and referential integrity validations are completed.

Before loading the Fact table, the process checks for **inferred members**. If a referenced dimension member does not exist, an inferred member is created in the corresponding dimension table with the IsInferred flag set to identify it as an inferred record. When the actual dimension data becomes available, the inferred member can be updated with the complete details.

Note: Inferred Member handling for DimProduct is currently disabled.

**3. Master and Child Package Execution**

A master SSIS package orchestrates the ETL process by executing separate child packages for the relevant tables and loading operations.

Each pipeline execution is treated as a **batch**, with a recorded start time, end time, and execution status.

- **Succeeded:** All required child packages and their data transfer operations complete successfully.
- **Failed:** At least one child package encounters a failure in a required data transfer operation.

**4. Logging and Batch Tracking**

Every table operation, including TRUNCATE, INSERT, and UPDATE, is recorded in the ETL execution log and linked to its corresponding batch.

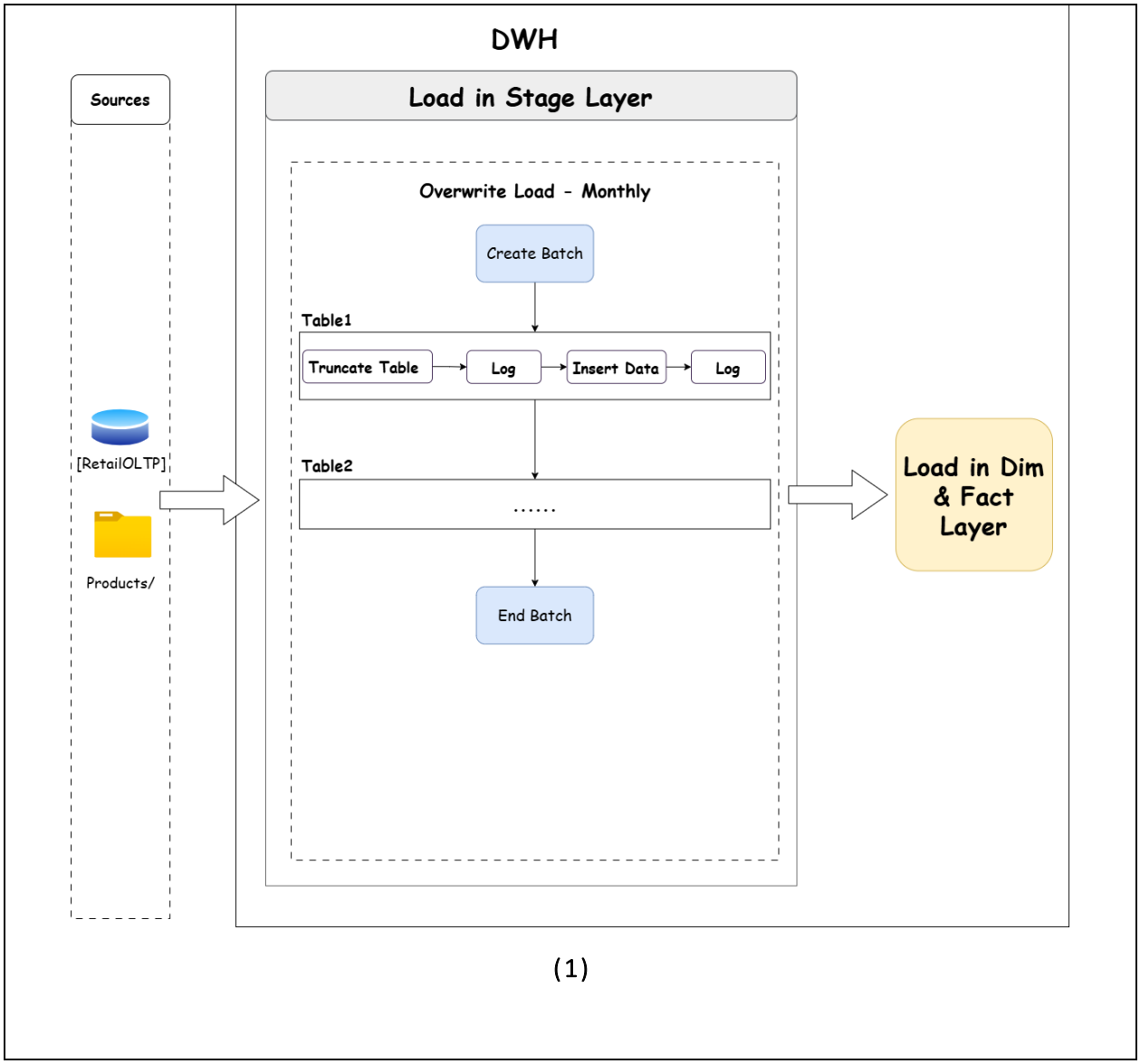
The execution log records the table name, operation type, load type, affected row count, success or failure status, and relevant messages.

The batch log tracks the overall execution period and status, which is determined by the outcomes of the child packages and their logged operations.

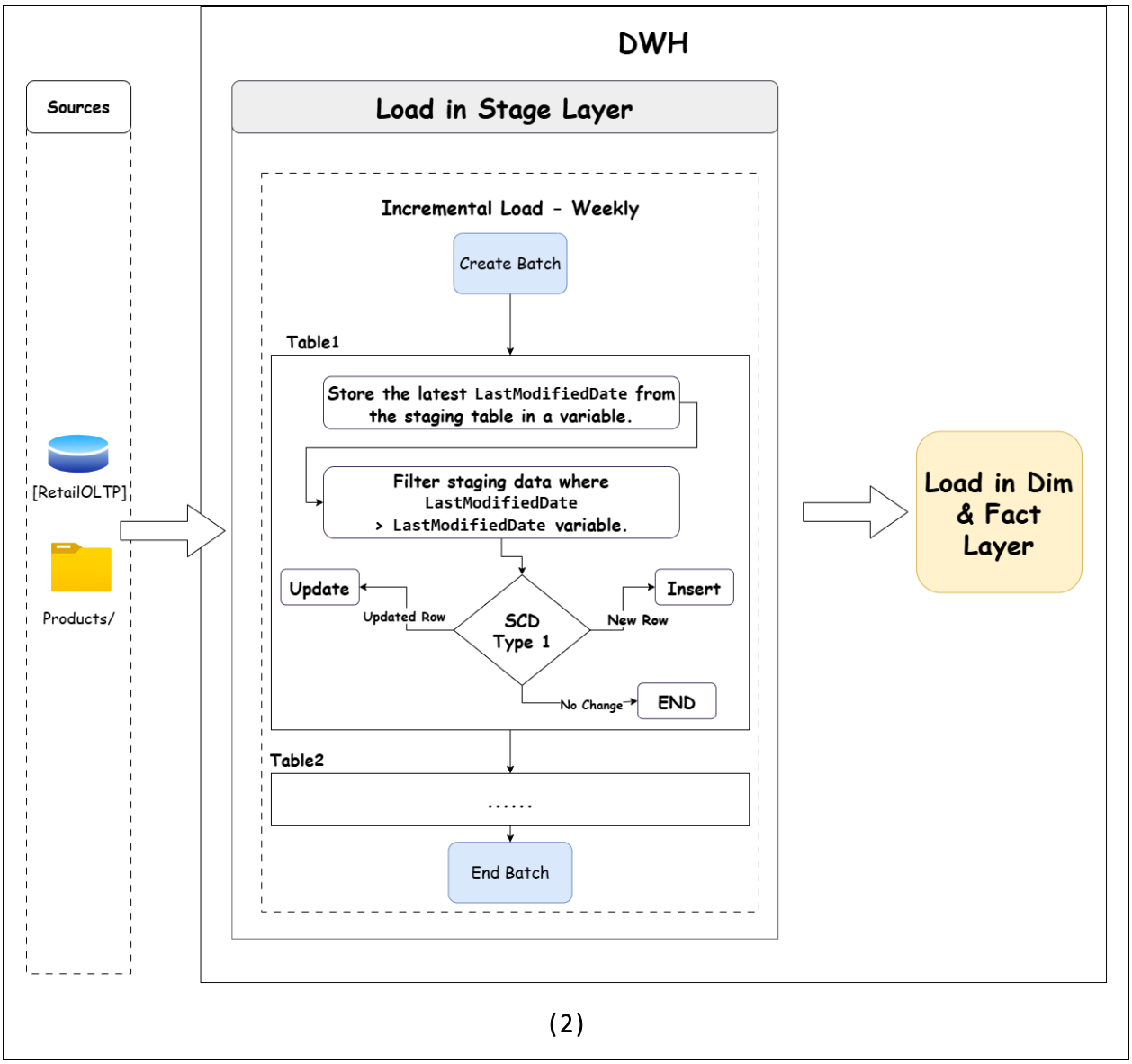
**5. Error Handling**

If a child package fails, the master package continues executing the remaining child packages independently. A failure in one package does not prevent the other packages from running.

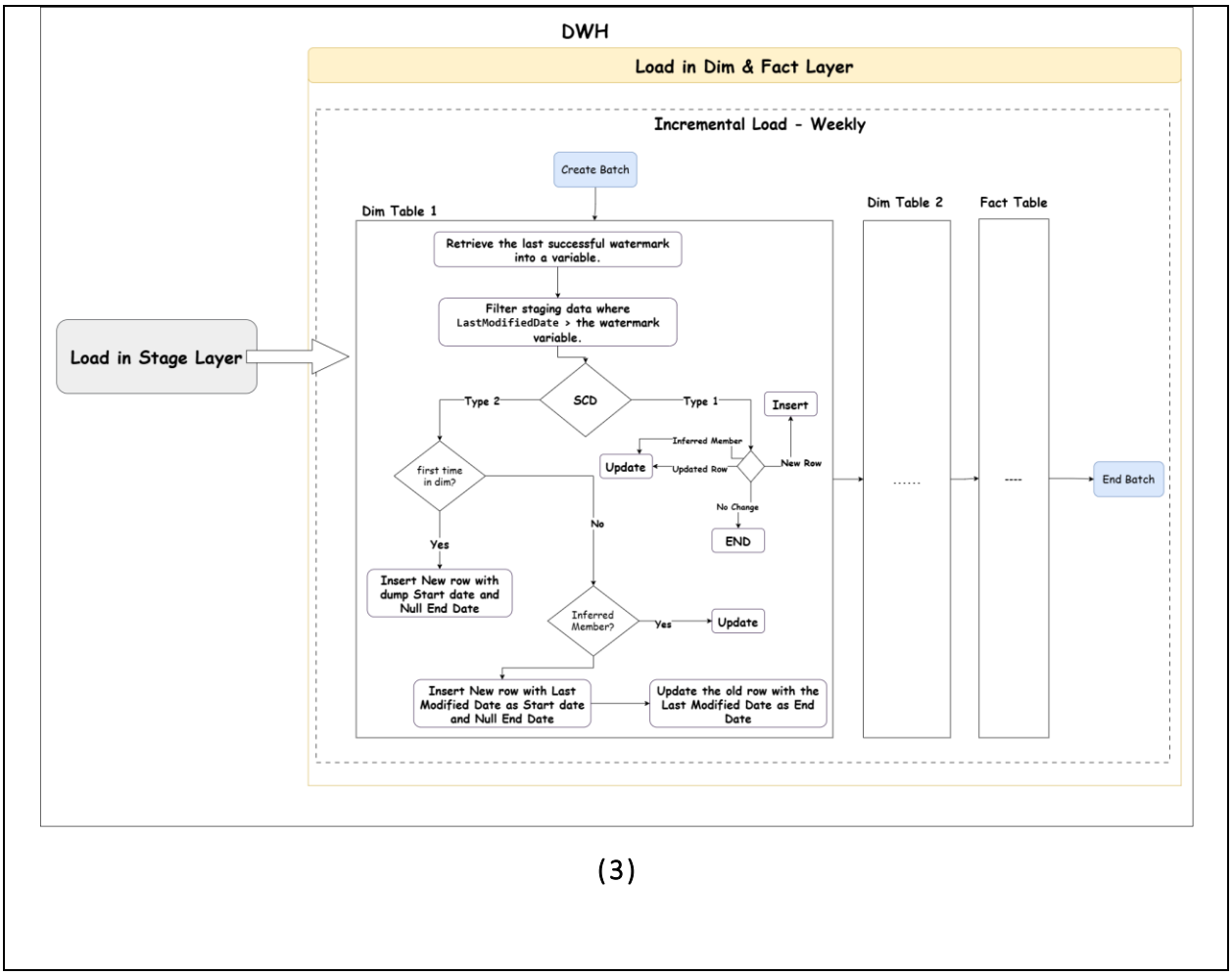
Failed operations are logged against the current batch, and the batch is marked as **Failed** if any required operation fails. Successful completion is recorded only when all required operations across the batch have completed successfully.



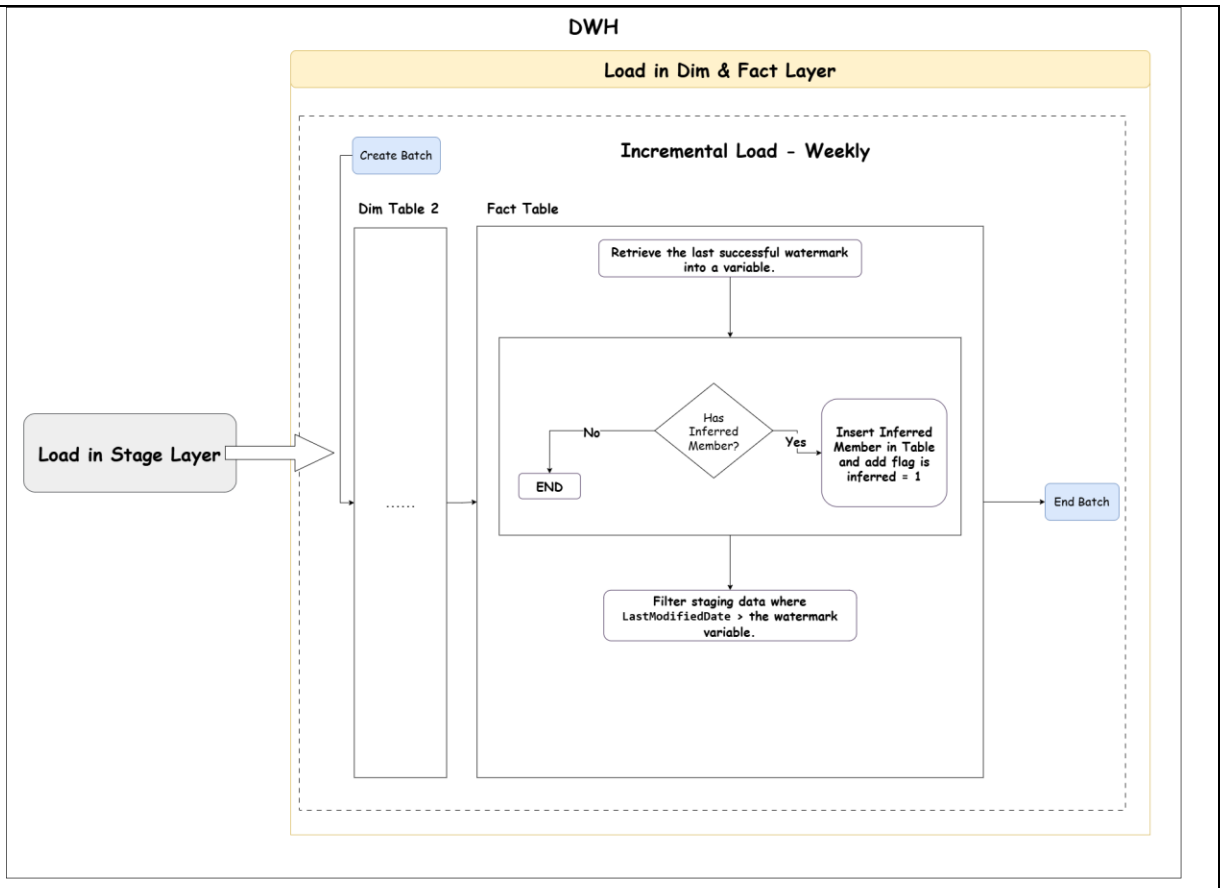

(1)


DWH
Sources
Load in Stage Layer
Incremental Load - Weekly
Create Batch
Table1
Store the latest LastModifiedDate from the staging table in a variable.
Filter staging data where LastModifiedDate > LastModifiedDate variable.
Update
Updated Row
SCD Type 1
New Row
Insert
No Change
END
Table2
.....
End Batch
[RetailOLTP]
Products/
Load in Dim & Fact Layer

(2)

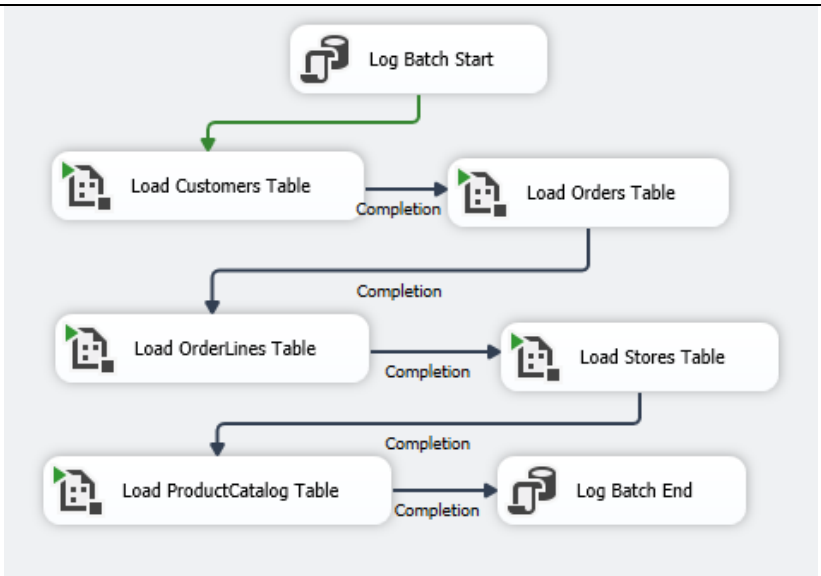

DWH
Load in Dim & Fact Layer
Incremental Load - Weekly
Load in Stage Layer
Create Batch
Dim Table 1
Retrieve the last successful watermark into a variable.
Filter staging data where LastModifiedDate > the watermark variable.
SCD
Type 2
Type 1
Insert
Inferred Member
Update
Updated Row
New Row
No Change
END
first time in dim?
No
Yes
Insert New row with dump Start date and Null End Date
Inferred Member?
Yes
Update
Insert New row with Last Modified Date as Start date and Null End Date
Update the old row with the Last Modified Date as End Date
Dim Table 2
Fact Table
.....
----
End Batch

(3)


DWH
Load in Dim & Fact Layer
Incremental Load - Weekly
Create Batch
Dim Table 2
Fact Table
Retrieve the last successful watermark into a variable.
Load in Stage Layer
No
Has Inferred Member?
Yes
Insert Inferred Member in Table and add flag is inferred = 1
END
End Batch
Filter staging data where LastModifiedDate > the watermark variable.

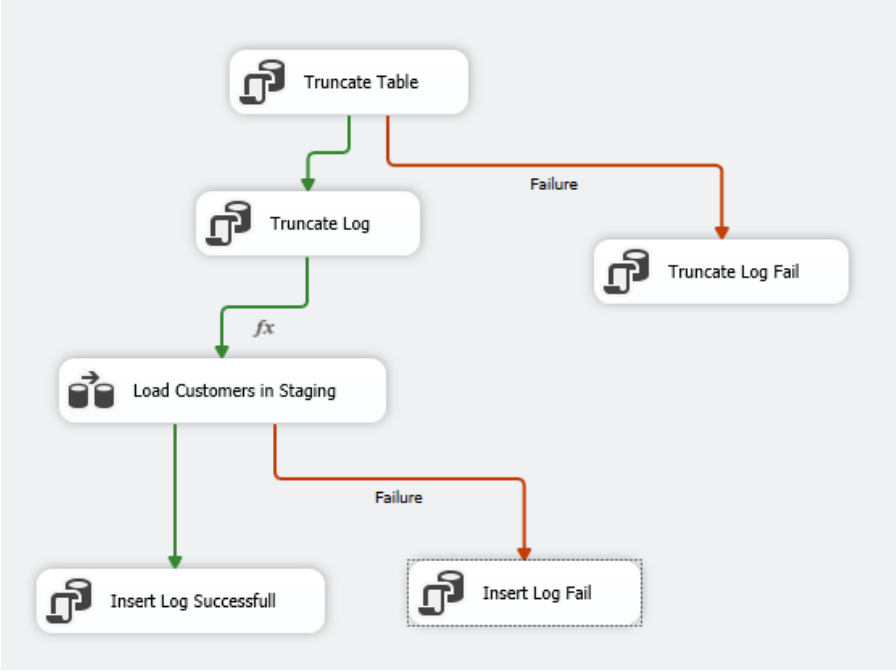
(4)

## Overwrite Stage Layer Execute Package:

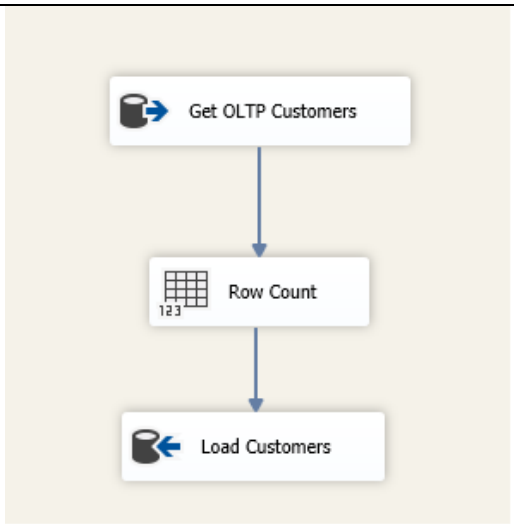



## Example Table Package

**Control Flow**




**Data Flow**




## Example Log Batch:

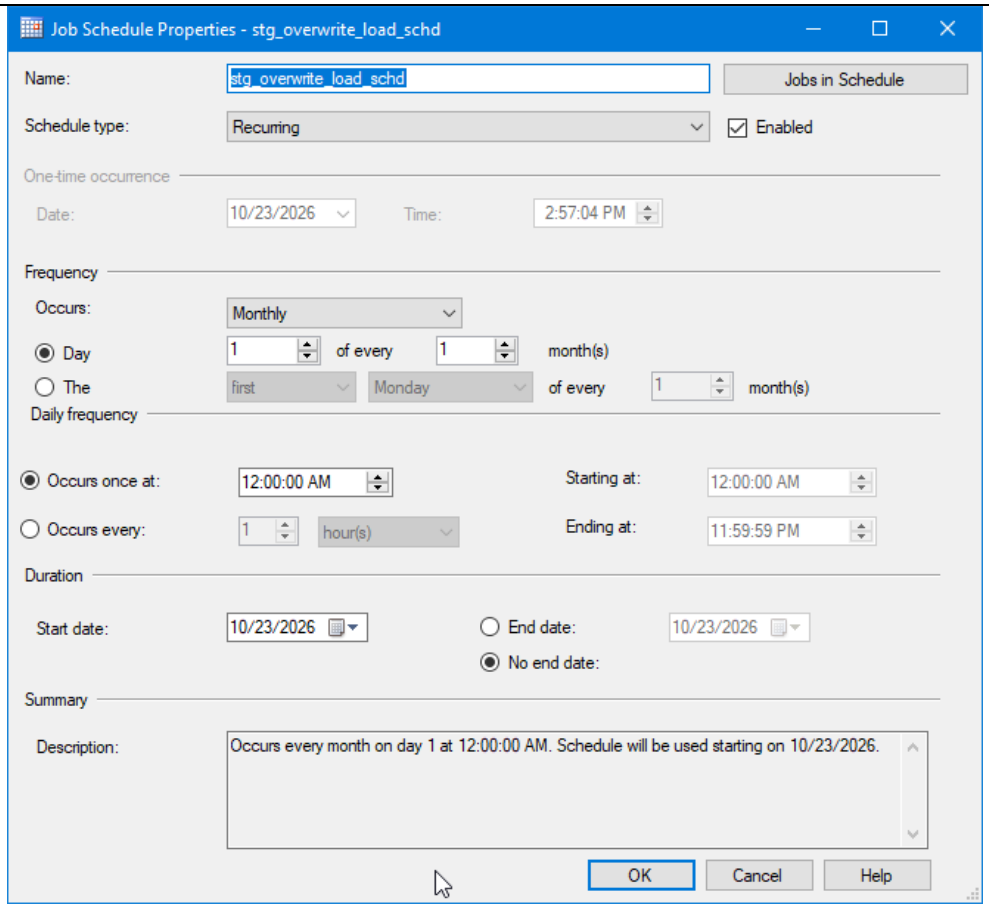
**Batch**

| | batch_id | layer_schema | start_date | end_date | status |
|---|---|---|---|---|---|
| 1 | 33 | stg | 2026-10-23 14:54:15.6633333 | 2026-10-23 14:54:31.6133333 | Succeeded |

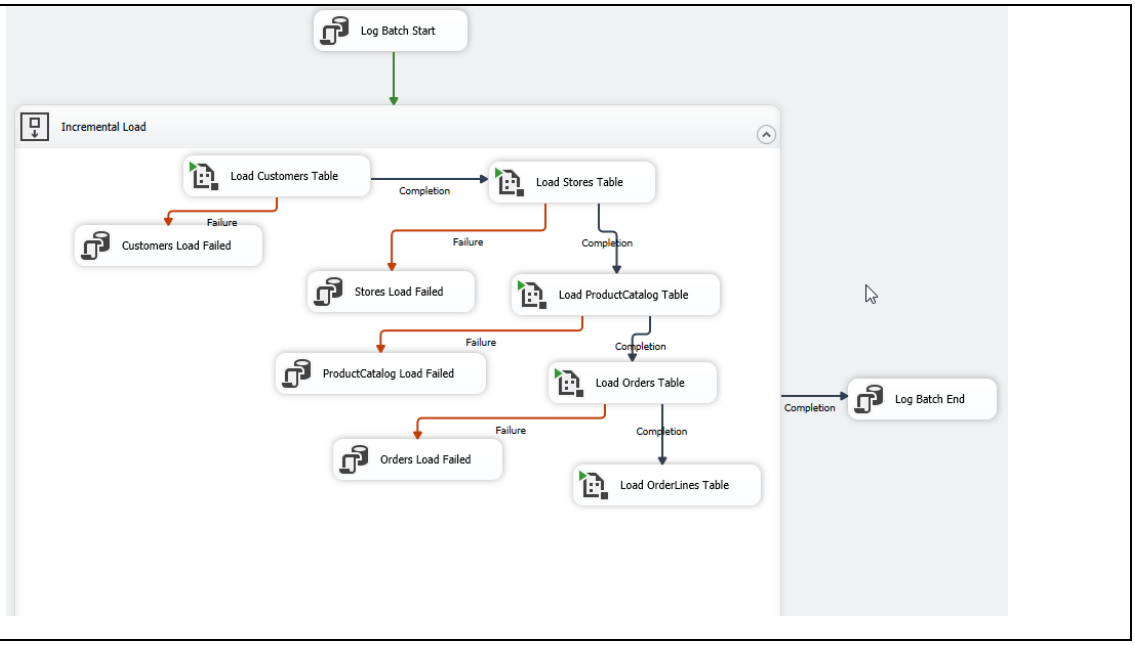
**Log operations in Batch**

| | log_id | batch_id | table_name | operation_type | load_type | no_rows | is_success | message | execution_date |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 263 | 33 | Customers | truncate | Overwrite | 0 | 1 | Truncate completed successfully. | 2026-10-23 14:54:17.333 |
| 2 | 264 | 33 | Customers | insert | Overwrite | 36 | 1 | Insert completed successfully. | 2026-10-23 14:54:19.080 |
| 3 | 265 | 33 | Orders | truncate | Overwrite | 0 | 1 | Truncate completed successfully. | 2026-10-23 14:54:20.413 |
| 4 | 266 | 33 | Orders | insert | Overwrite | 181 | 1 | Insert completed successfully. | 2026-10-23 14:54:22.097 |
| 5 | 267 | 33 | OrderLines | truncate | Overwrite | 0 | 1 | Truncate completed successfully. | 2026-10-23 14:54:23.963 |
| 6 | 268 | 33 | OrderLines | insert | Overwrite | 470 | 1 | Insert completed successfully. | 2026-10-23 14:54:25.350 |
| 7 | 269 | 33 | Stores | truncate | Overwrite | 0 | 1 | Truncate completed successfully. | 2026-10-23 14:54:27.350 |
| 8 | 270 | 33 | Stores | insert | Overwrite | 6 | 1 | Insert completed successfully. | 2026-10-23 14:54:28.920 |
| 9 | 271 | 33 | ProductCatalog | truncate | Overwrite | 0 | 1 | Truncate completed successfully. | 2026-10-23 14:54:30.350 |
| 10 | 272 | 33 | ProductCatalog | insert | Overwrite | 25 | 1 | Insert completed successfully. | 2026-10-23 14:54:31.557 |

## Job:

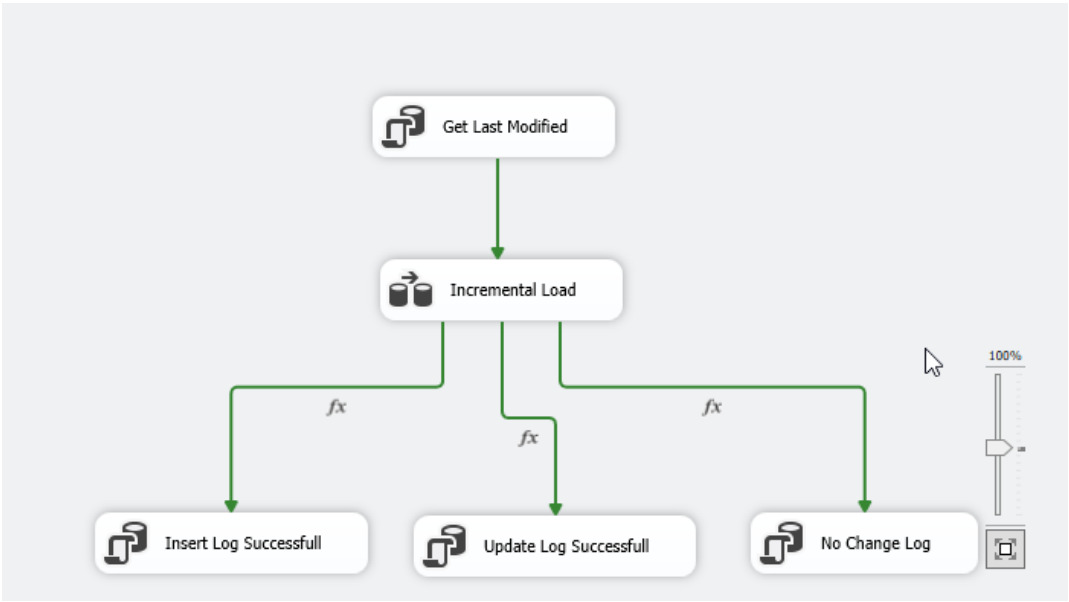

Job Schedule Properties - stg_overwrite_load_schd
Name:
stg_overwrite_load_schd
Jobs in Schedule
Schedule type:
Recurring
Enabled
One-time occurrence
Date:
10/23/2026
Time:
2:57:04 PM
Frequency
Occurs:
Monthly
Day
1
of every
1
month(s)
The
first
Monday
of every
1
month(s)
Daily frequency
Occurs once at:
12:00:00 AM
Starting at:
12:00:00 AM
Occurs every:
1
hour(s)
Ending at:
11:59:59 PM
Duration
Start date:
10/23/2026
End date:
10/23/2026
No end date:
Summary
Description:
Occurs every month on day 1 at 12:00:00 AM. Schedule will be used starting on 10/23/2026.
OK
Cancel
Help

## Incremental Stage Layer Execute Package:

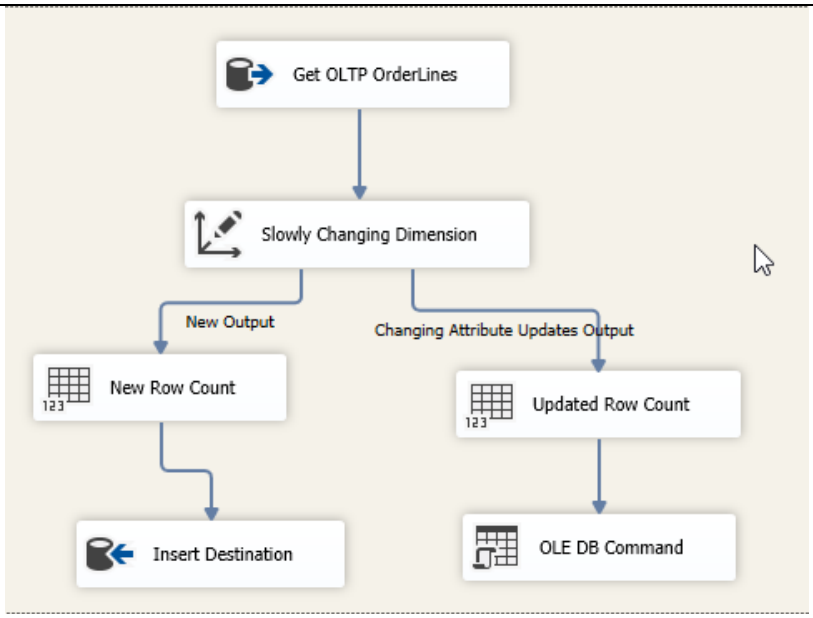



## Example Table Package

**Control Flow**




**Data Flow**




## Example Log Batch:

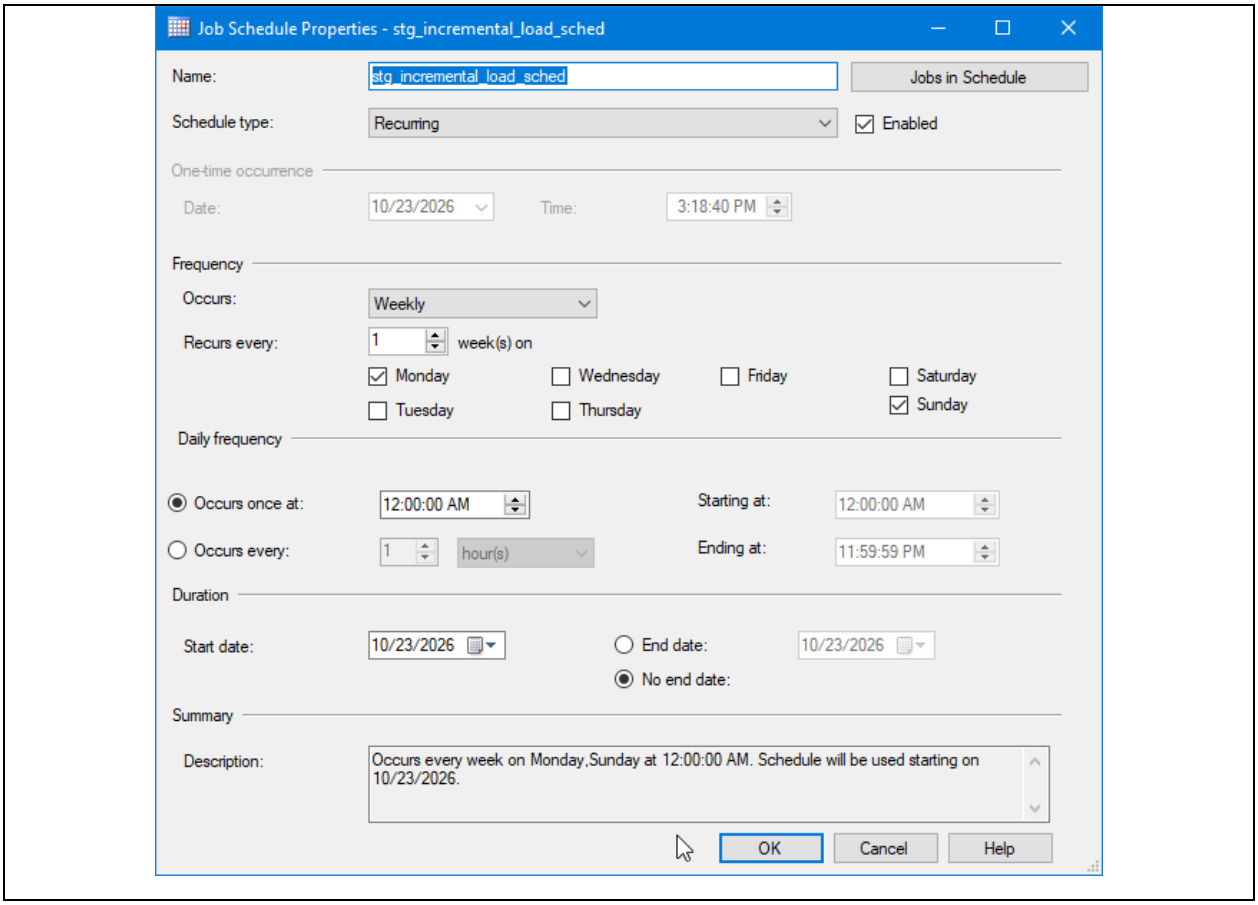
**Batch**

| | batch_id | layer_schema | start_date | end_date | status |
|---|---|---|---|---|---|
| 1 | 35 | stg | 2026-10-23 15:02:26.6000000 | 2026-10-23 15:03:06.1833333 | Succeeded |

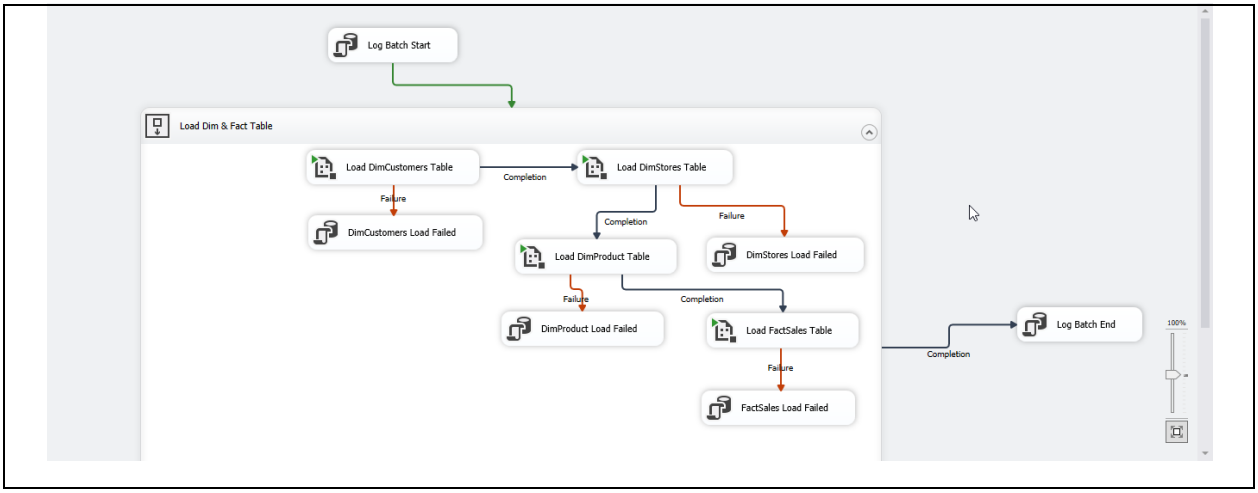
**Log operations in Batch**

| | log_id | batch_id | table_name | operation_type | load_type | no_rows | is_success | message | execution_date |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 283 | 35 | Customers | NULL | Incremental | 0 | 1 | No changes detected | 2026-10-23 15:02:30.047 |
| 2 | 284 | 35 | Stores | NULL | Incremental | 0 | 1 | No changes detected | 2026-10-23 15:02:40.773 |
| 3 | 285 | 35 | ProductCatalog | NULL | Incremental | 0 | 1 | No changes detected | 2026-10-23 15:02:51.767 |
| 4 | 286 | 35 | Orders | NULL | Incremental | 0 | 1 | No changes detected | 2026-10-23 15:03:02.773 |
| 5 | 287 | 35 | OrderLines | NULL | Incremental | 0 | 1 | No changes detected | 2026-10-23 15:03:06.053 |

## Job:

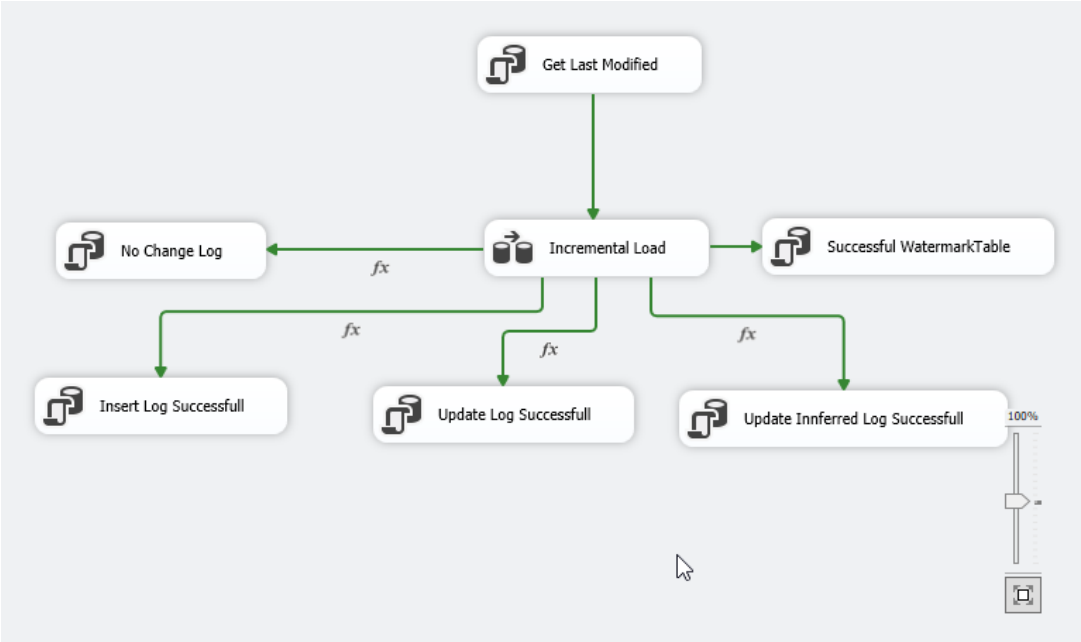



## Incremental dim & fact Layer Execute Package:

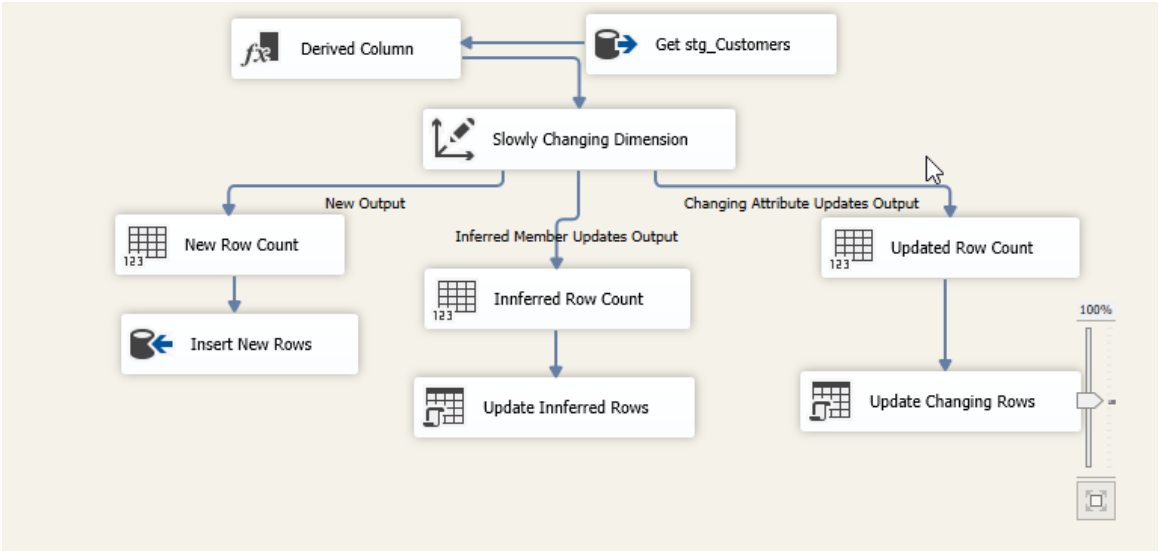


## Example Dim Table Package SCD1

**Control Flow**

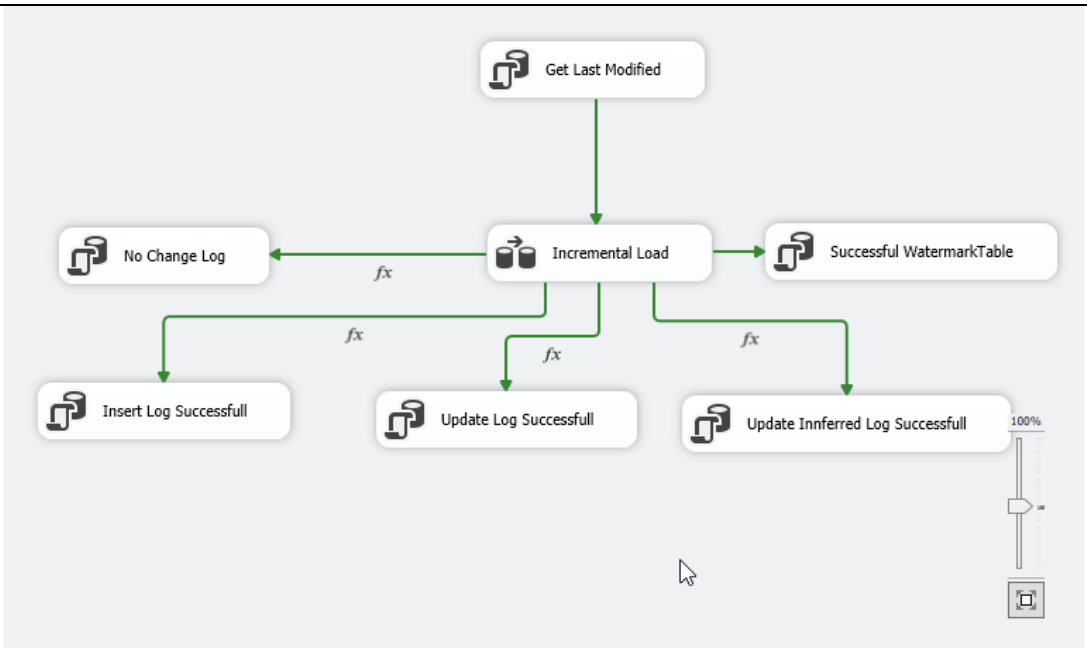



**Data Flow**

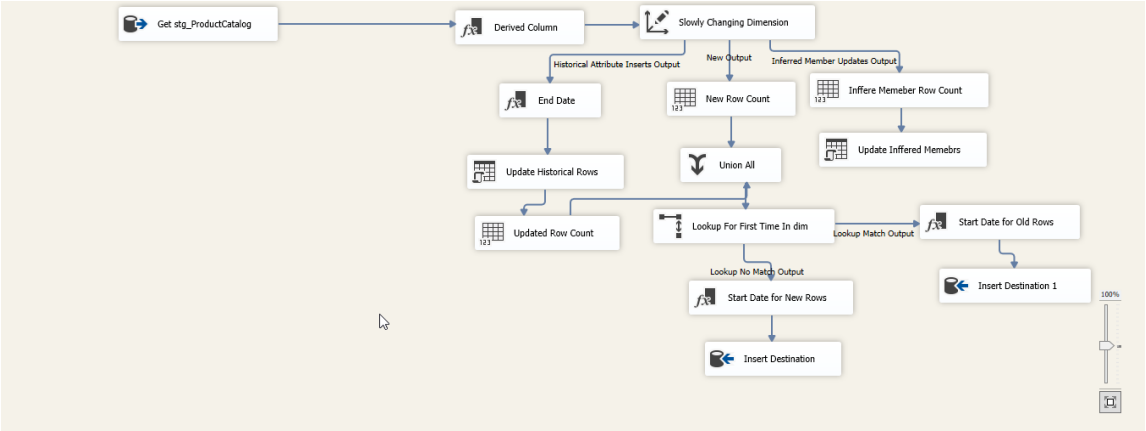



## Example Dim Table Package SCD2:

**Control Flow**

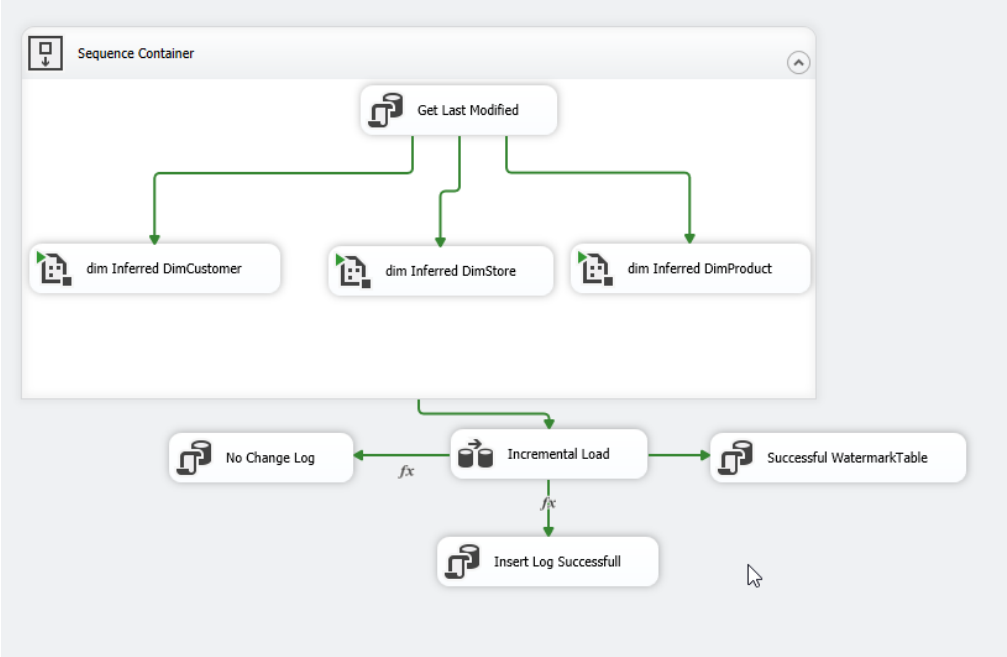



**Data Flow**

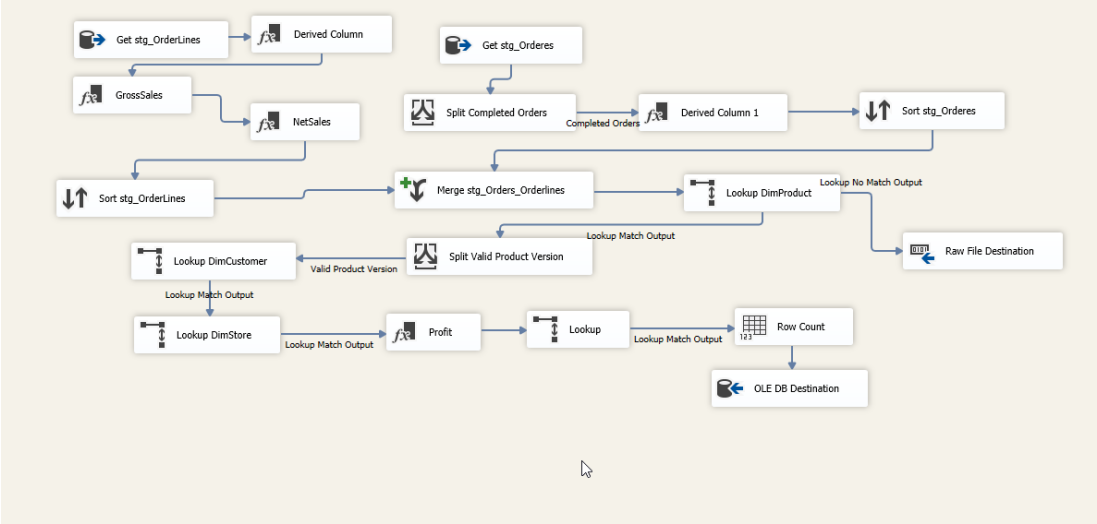



## Example Fact Table Package:

**Control Flow**

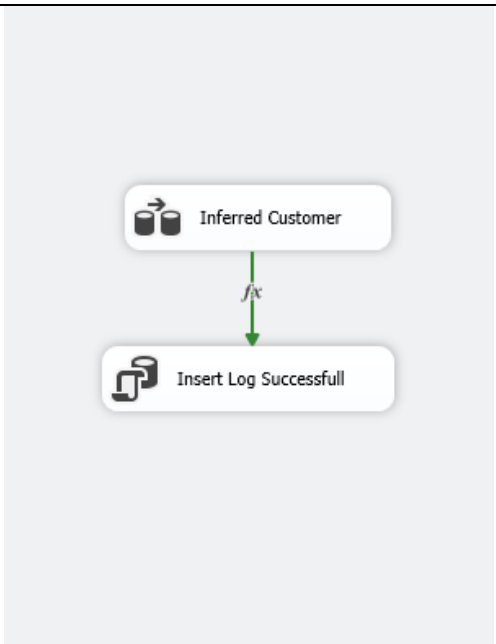



**Data Flow**

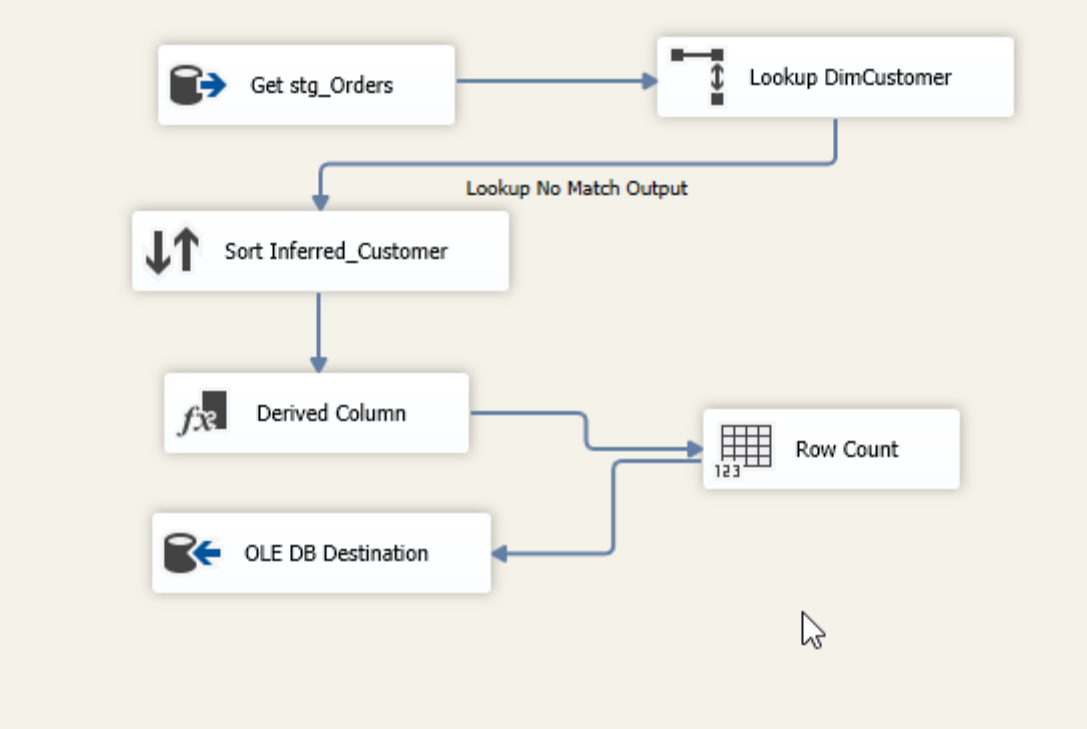



## Example Inferred Member Package:

**Control Flow**


Inferred Customer
Insert Log Successfull

## Data Flow


Get stg_Orders
Lookup DimCustomer
Lookup No Match Output
Sort Inferred_Customer
Derived Column
Row Count
OLE DB Destination

## Example Log Batch:

**Batch**

| | batch_id | layer_schema | start_date | end_date | status |
|---|---|---|---|---|---|
| 1 | 40 | dim & fact | 2026-10-23 16:15:17.6633333 | 2026-10-23 16:15:45.9466667 | Succeeded |

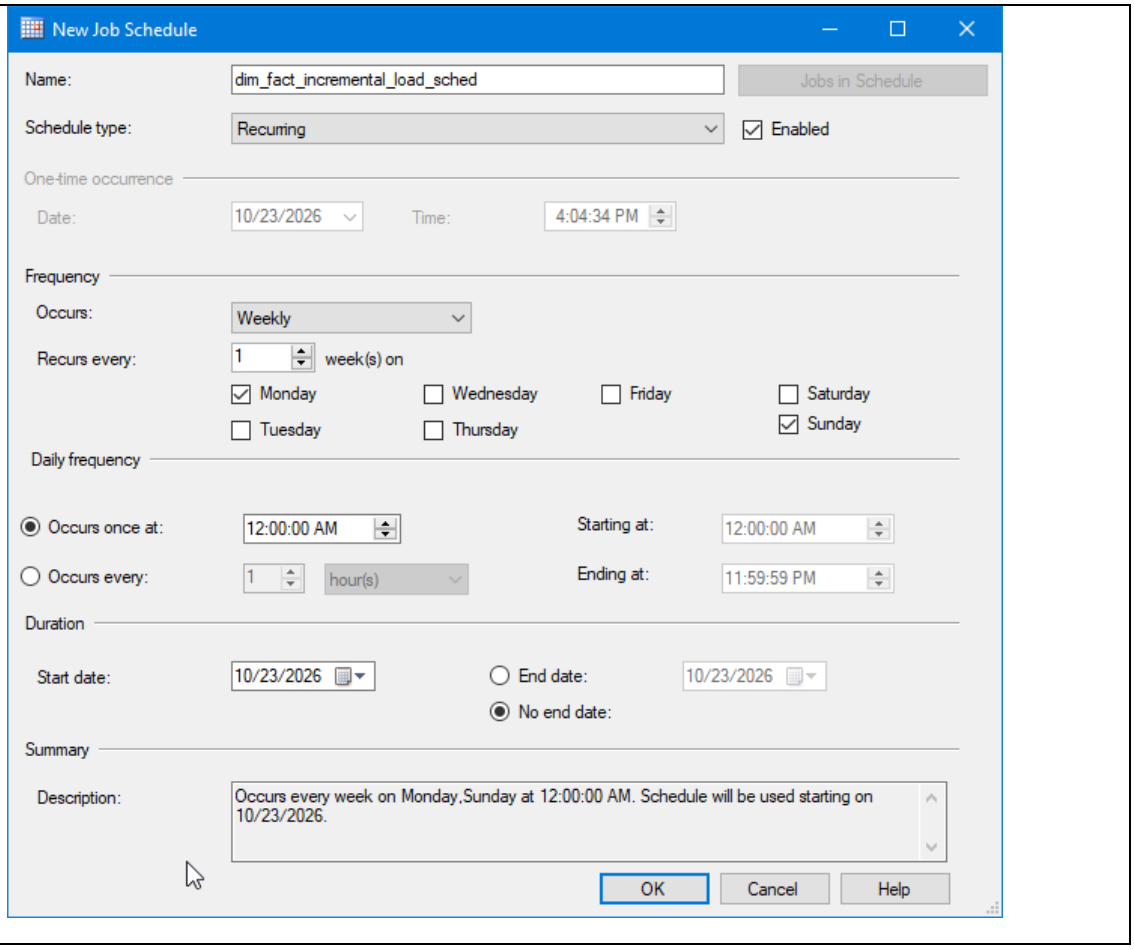
**Log operations in Batch**

| | log_id | batch_id | table_name | operation_type | load_type | no_rows | is_success | message | execution_date |
|---|---|---|---|---|---|---|---|---|---|
| 300 | 302 | 40 | DimCustomer | insert | Increme... | 36 | 1 | Insert completed successfully. | 2026-10-23 16:15:19.807 |
| 301 | 303 | 40 | DimStore | insert | Increme... | 6 | 1 | Insert completed successfully. | 2026-10-23 16:15:20.963 |
| 302 | 304 | 40 | DimProduct | insert | Increme... | 25 | 1 | Insert completed successfully. | 2026-10-23 16:15:23.177 |
| 303 | 305 | 40 | DimProduct | insert | Increme... | 1 | 1 | Insert Inffered Member complet... | 2026-10-23 16:15:44.053 |
| 304 | 306 | 40 | FactSales | insert | Increme... | 431 | 1 | Insert completed successfully. | 2026-10-23 16:15:45.883 |

## Watermarks:

| | TableName | LastSuccessfulWatermark |
|---|---|---|
| 1 | DimCustomer | 2026-10-23 16:15:19.797 |
| 2 | DimProduct | 2026-10-23 16:15:23.163 |
| 3 | DimStore | 2026-10-23 16:15:20.950 |
| 4 | FactSales | 2026-10-23 16:15:45.867 |

## Job:

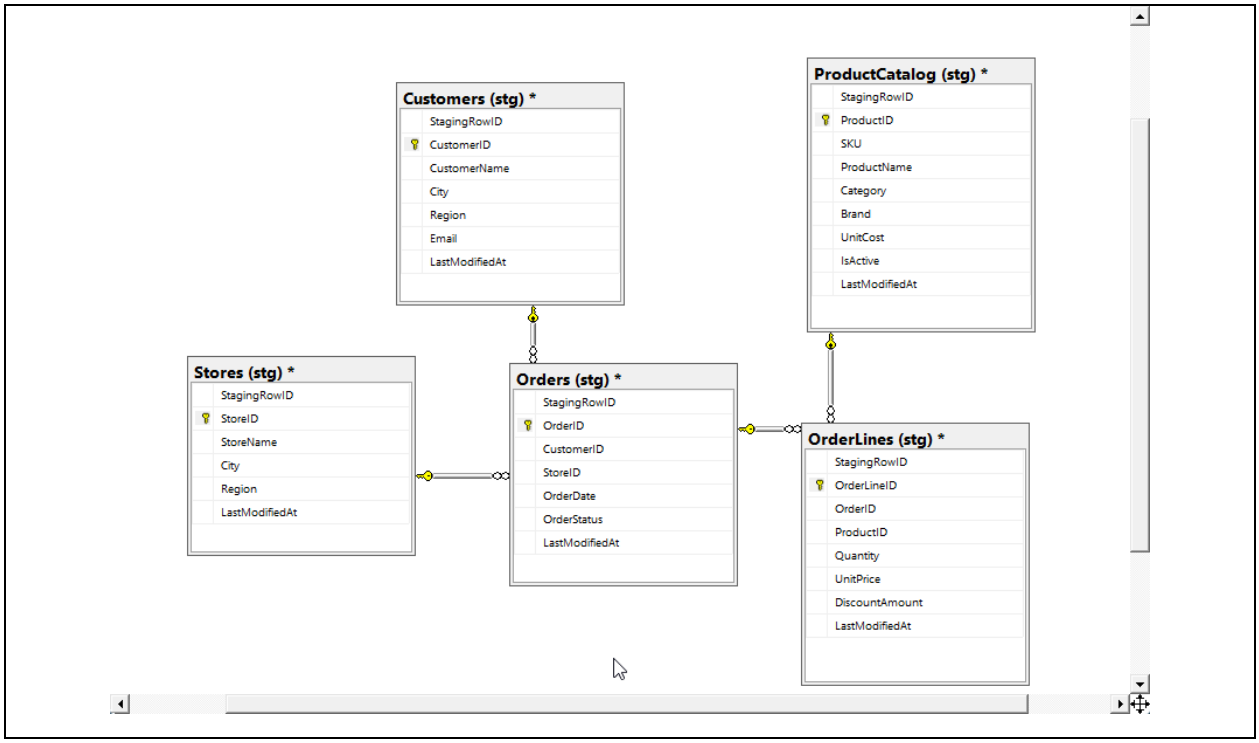



# Data Layers:

- ## Stage Layer:
  - ### Data Dictionary:

- **Data Loading:** Data is transferred from the source systems to the staging tables without any modifications or transformations.
- **StagingRowID**: An IDENTITY column is added to each staging table as a technical identifier for staging records only. It is not a surrogate key used in the data warehouse dimensions.

### Tables:




## Dim and Fact Layer:

- **Business Keys vs. Surrogate Keys**: CustomerID, StoreID, ProductID, and DateKey identify business records, while CustomerKey, StoreKey, ProductKey, and DateKey serve as dimension keys in the warehouse. SCD Type 2 allows multiple ProductKey values for the same ProductID to preserve historical versions.

### DimCustomer Data Dictionary:

- **Source:** Data is loaded from stg.Customers.
- **Surrogate Key:** CustomerKey, generated as an identity column.
- **Column Mapping:** All other columns retain their original names and are mapped from the source table without renaming.
- **SCD Type:** Type 1 is applied during loading to update existing records with the latest values without maintaining historical versions.

### DimStore Data Dictionary:

- **Source:** Data is loaded from stg.Stores.
- **Surrogate Key:** StoreKey, generated as an identity column.

- **Column Mapping:** All other columns retain their original names and are mapped from the source table without renaming.
- **SCD Type:** Type 1 is applied during loading to update existing records with the latest values without maintaining historical versions.

### DimProduct Data Dictionary:

- **Source:** Data is loaded from stg.ProductCatalog.
- **Surrogate Key:** ProductKey, generated as an identity column.
- **Column Mapping:** Product attributes are mapped from the source table. ProductID is not unique because each product can have multiple historical versions.

**SCD Type:** Type 2 (SCD2) is applied during loading to preserve historical product versions, particularly changes in product prices.

- **StartDate:** For a product loaded into the dimension for the first time, a default date (1900-01-01) is assigned. For a new version of an existing product, StartDate is set to the source record's LastModifiedAt value.
- **EndDate:** When an existing product changes, the previous version's EndDate is set to the LastModifiedAt value of the new version. The newly inserted version receives this same value as its StartDate, while its EndDate is set (NULL) to indicate that it is the current version.

These dates allow each sales transaction to be matched to the appropriate product version and unit cost for its order date, supporting accurate historical profit calculations.

Use non-overlapping time intervals: StartDate <= event time < EndDate. Set the current row's EndDate to a far-future value. Enforce this date-range logic using a CHECK constraint on the dimension table.

### FactSales Data Dictionary:

- **Grain**: One row per order line.
- **Order Status Filter**: Only orders with a Completed status are loaded into FactSales and included in revenue calculations. Pending and Cancelled orders are excluded.
- **Foreign Key Constraints:** Foreign key constraints are defined on FactSales to enforce referential integrity between the fact table and its related dimension tables.
- **One-to-Many Dimension Relationships**: Each dimension record can be associated with multiple records in FactSales, while each fact record references the appropriate dimension record through its foreign key.
- **Source:** Data is loaded from stg.Orders and stg.OrderLines.
- **Surrogate Key:** SalesKey, generated as an identity column.

- **Column Mapping:** Order and order-line attributes are mapped from their respective staging tables, with dimension surrogate keys used to establish relationships with the dimension tables.
- **SCD Type:** No SCD is applied to the fact table. Data is loaded incrementally.
- **Degenerate Dimension:** OrderID is stored directly in FactSales as a degenerate dimension, without creating a separate DimOrder table.

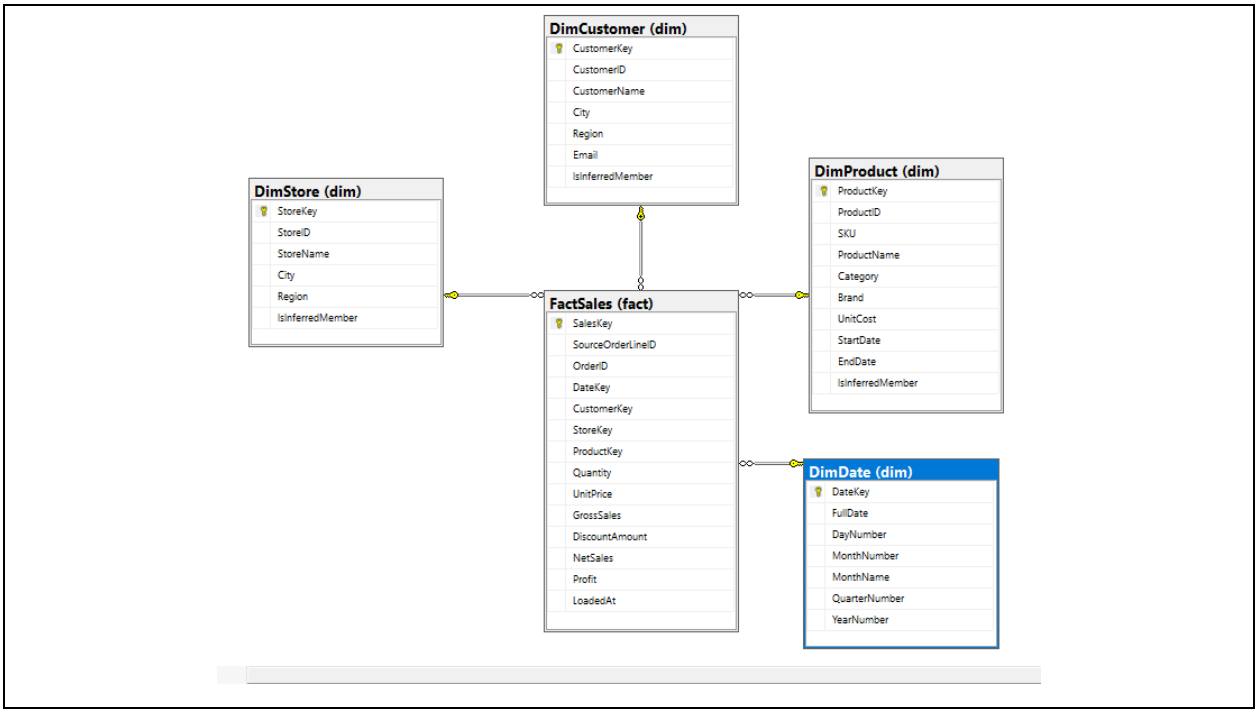
**Derived Columns:**

- GrossSales: Calculated as Quantity * UnitPrice.
- NetSales: Calculated as GrossSales - DiscountAmount.
- Profit: Calculated as NetSales - (Quantity * UnitCost), using the appropriate historical product cost based on the order date.

**Unknown and Inferred Member Handling**

- **DimCustomer and DimStore:** An Unknown Member with a surrogate key of 0 is maintained in each dimension to handle missing or unavailable customer and store information. When a fact record contains a NULL or 0 dimension key, it is assigned to the corresponding Unknown Member. If the key contains another value that does not exist in the dimension table, an Inferred Member is created with the IsInferredMember flag set to 1.
- **DimProduct:** If an order line references a product that does not exist in DimProduct, the record is redirected to an output file for further investigation rather than being assigned an arbitrary ProductKey. This ensures that missing product references are identified and handled explicitly before loading the fact table.

- Data Modeling:




## Technologies Used:

- **SQL Server:** Database management and data warehouse implementation.
- **T-SQL:** Data cleaning, transformation, business rules, and stored procedures.
- **SQL Server Management Studio (SSMS):** Database development, query execution, and data validation.
- **SQL Server Integration Services (SSIS):** ETL package development, orchestration, staging table loading, and error handling.
- **draw.io**: Documentation of ETL workflows, data warehouse architecture, and data flow diagrams.

## Links & References:

- **LinkedIn:** [www.linkedin.com/in/roqiua-mohammad](www.linkedin.com/in/roqiua-mohammad)
- **GitHub Repository:** [https://github.com/roqiua/RetailDWH_bootcamp_databill](https://github.com/roqiua/RetailDWH_bootcamp_databill)